

PHẦN 1: KIẾN TRÚC TỔNG THỂ (TECH STACK)

Mô hình Full-stack Serverless quy tụ toàn bộ dự án về một mối, giảm thiểu tối đa chi phí vận hành và rào cản kết nối hệ thống.

Layer	Công nghệ	Chi tiết & Vai trò
Frontend Framework	Next.js (React)	Xử lý giao diện người dùng. Tận dụng cơ chế SSR (Server-Side Rendering) giúp SEO sản phẩm cực tốt. Hỗ trợ chuyển đổi nhanh chóng các file HTML/CSS hiện có sang Component (JSX/TSX).
UI & Styling	Tailwind CSS	Định hình phong cách thiết kế của thương hiệu (minimalist, premium).
Backend Logic	Next.js Route Handlers	Toàn bộ API của dự án sẽ được viết trực tiếp trong Next.js (thư mục <code>app/api/</code>) dưới dạng Serverless Functions. Không cần khởi tạo server Node.js hay Django riêng lẻ.
Database & Auth	Supabase (PostgreSQL)	Đóng vai trò là trung tâm dữ liệu. Cung cấp sẵn Database PostgreSQL, hệ thống Authentication (Đăng nhập/Đăng ký) và Cloud Storage để lưu trữ hình ảnh nền tùy chỉnh.
Database ORM	Prisma	Công cụ giao tiếp với Database. PM sẽ thiết kế cấu trúc dữ liệu qua file <code>schema.prisma</code> , giúp thao tác với PostgreSQL trực quan, an toàn và có typing chặt chẽ.
Deployment	Vercel	Nơi host duy nhất cho toàn bộ source code Next.js (cả Frontend lẫn Backend Serverless API).

PHẦN 2: PHÂN CHIA KHÔNG GIAN LÀM VIỆC (WORKSPACE)

Dù code chung một Repo, PM cần quy định rõ "lãnh địa" của từng Dev để quá trình làm việc song song diễn ra trơn tru.

Cấu trúc thư mục định hướng (Next.js App Router):

Plaintext

chamcham-studio/

```
|— app/
|   |— (frontend)/    <-- DEV FE: Chứa các page giao diện (Trang chủ, Sản phẩm, Giỏ hàng)
|   |— api/          <-- BE: Chứa các file Route Handlers (API endpoint)
|   |— components/   <-- FE: Chứa các UI Components dùng chung (Button, Modal)
|   |— layout.tsx
|— prisma/
|   |— schema.prisma  <-- PM: Nơi PM thiết kế Database, BE Dev sử dụng để query
|— public/           <-- Chứa hình ảnh tĩnh, fonts
|— .env              <-- PM cung cấp các biến môi trường (Supabase keys)
```

- **Trách nhiệm của PM:** Cấu hình Vercel, setup project Supabase, thiết kế cấu trúc các bảng trong `schema.prisma`, và quản lý tiến độ.
- **Trách nhiệm của Dev FE:** Chuyển đổi HTML/CSS sang React Components, xử lý State (giỏ hàng, tùy chỉnh nền), gọi API từ thư mục `app/api` để lấy/gửi dữ liệu.
- **Trách nhiệm của Dev BE:** Viết logic xử lý trong `app/api`, kết nối với Supabase qua Prisma để tương tác với Database (tạo đơn hàng, xác thực user, lấy danh sách sản phẩm).

PHẦN 3: QUY TẮC GIT VÀ LUỒNG TRIỂN KHAI (GIT WORKFLOW)

Với Monorepo, nếu FE và BE cùng sửa đổi một file (ví dụ `package.json` hoặc các file config), conflict rất dễ xảy ra. Áp dụng nghiêm ngặt các quy tắc sau:

1. Chiến lược phân nhánh (Branching)

Hệ thống sử dụng 3 cấp độ nhánh:

- **main (Production):** Chỉ chứa code đã hoàn thiện, deploy thẳng ra môi trường thật. **Chỉ PM có quyền Merge.**
- **develop (Test/Preview):** Môi trường để gom code của FE và BE lại để test luồng thực tế. **Chỉ PM có quyền Merge thông qua Pull Request (PR).**
- **feature/... hoặc bugfix/...:** Nhánh làm việc riêng của từng Dev. Bắt buộc tạo từ nhánh `develop`.
 - *Dev FE tạo:* `feature/fe-shopping-cart`, `feature/fe-product-detail`
 - *Dev BE tạo:* `feature/be-api-create-order`, `feature/be-auth-login`

2. Quy ước Commit Code

Sử dụng tiền tố rõ ràng để PM đọc log là hiểu file nào vừa bị thay đổi:

- **fe: [Nội dung]** -> Cập nhật liên quan đến Frontend (UI, React components).
- **be: [Nội dung]** -> Cập nhật liên quan đến Backend (Route Handlers, API logic).

- **db: [Nội dung]** -> Cập nhật liên quan đến Prisma Schema (Thường do PM thực hiện).
- **chore: [Nội dung]** -> Các thay đổi cấu hình, cài đặt thêm thư viện.

3. Quy trình vận hành hằng ngày

1. **Đồng bộ:** Mỗi sáng, 2 Dev bắt buộc kéo code mới nhất từ **develop** về: **git pull origin develop**.
2. **Code tách biệt:** FE chỉ làm việc trong **app/(frontend)** và **components/**. BE chỉ làm việc trong **app/api/**.
3. **Tạo Pull Request:** Khi xong task, Dev push nhánh của mình lên GitHub và mở Pull Request (PR) trở vào nhánh **develop**.
4. **Review & Merge:** PM sẽ vào check PR. Nhờ cơ chế Vercel Preview, mỗi PR sẽ sinh ra một đường link test riêng biệt. PM click vào link đó nghiệm thu, nếu đạt chuẩn mới nhấn **Merge** vào **develop**.
5. **Release:** Sau 1-2 tuần, khi tính năng trên **develop** chạy ổn định, PM tạo PR gộp **develop** vào **main** để đưa lên môi trường Production phục vụ người dùng.